Intel® XeSS Plugin for Unreal Engine

Introduction

Intel® XeSS Plugin for Unreal Engine integrates Intel® XeSS 2 technologies into Unreal Engine (UE). XeSS Super Resolution (XeSS-SR), XeSS Frame Generation (XeSS-FG) and Xe Low Latency (XeLL) are parts of XeSS 2.

For more information, please visit: https://github.com/intel/xess

Installing the plugin

1. If you use the source code version of UE, you need to build the plugin locally, otherwise skip this step.

- Open the terminal and go to `<ue_root_dir>\Engine\Build\BatchFiles`
- Run the command line:

```
RunUAT.bat BuildPlugin -Plugin="<path_to_downloaded_plugin>\XeSS.uplugin" -Package="
<path_to_destination_plugin>" -TargetPlatforms=Win64 -VS2019
```

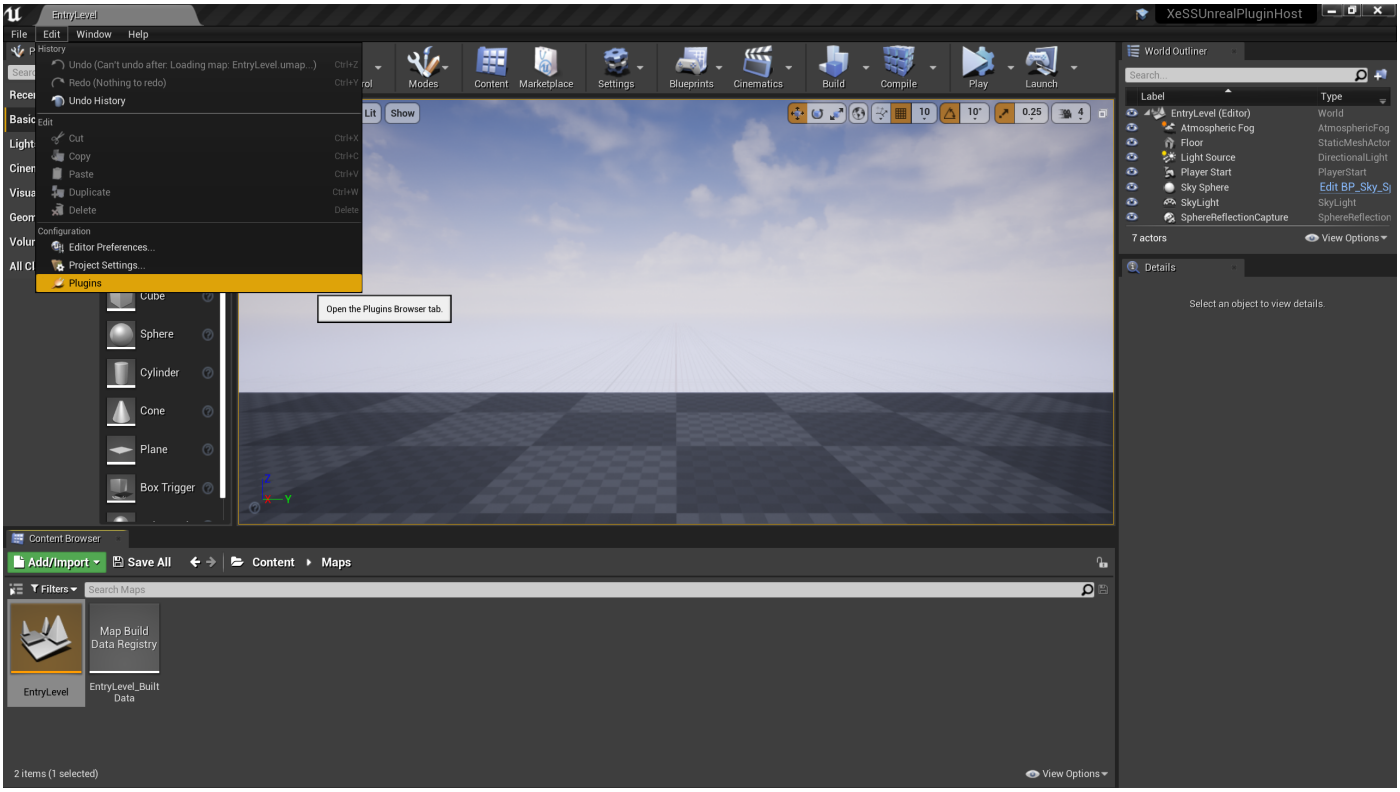
> The last parameter is the version of Visual Studio used to build, required by UE 4 only.

2. Copy (pre-)built files to:

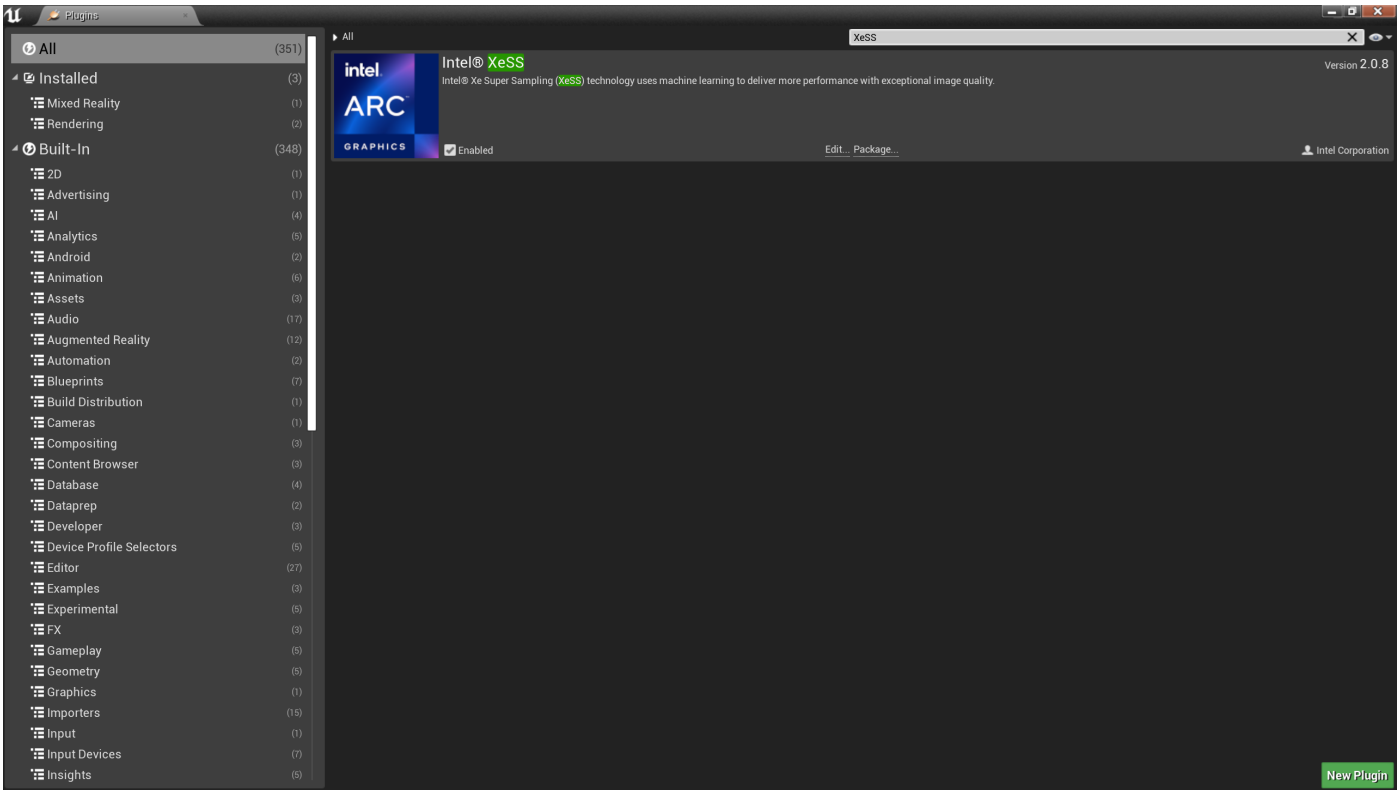
- For UE 4: `<ue_root_dir>\Engine\Plugins\Runtime\Intel\XeSS`
- For UE 5: `<ue_root_dir>\Engine\Plugins\Marketplace\XeSS`

Enabling the plugin in the Editor

To enable the XeSS plugin for a project, go to the plugin selection.



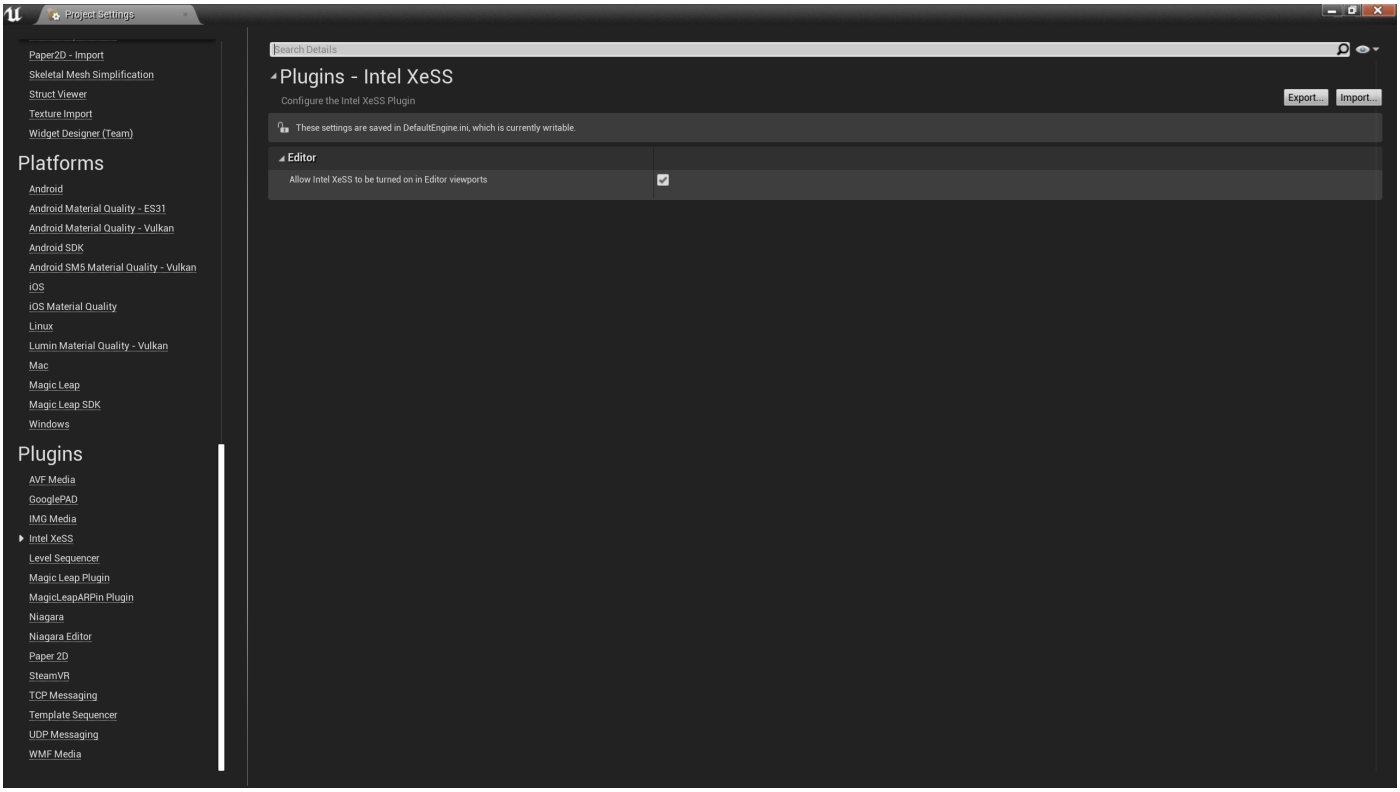
Use the search box to find the XeSS plugin and make sure it is enabled.



Project settings

Once this plugin is enabled, its default behavior can be modified in the Project Settings menu.

It is possible to disable/enable it in the Editor viewports.



Project packaging

No additional steps are needed for packaging.

Localization support

Localized text is supported via string table "XeSSStringTable", which can be referenced in widgets, Blueprint and C++ code.

Currently these cultures are supported:

Abbreviation	Name
ar-SA	Arabic (Saudi Arabia)
da-DK	Danish (Denmark)
de-DE	German (Germany)
en	English
es-ES	Spanish (Spain)
fr-FR	French (France)
it-IT	Italian (Italy)
ja-JP	Japanese (Japan)
ko-KR	Korean (South Korea)
nl-NL	Dutch (Netherlands)
pl-PL	Polish (Poland)
pt-PT	Portuguese (Portugal)
ru-RU	Russian (Russia)
uk-UA	Ukrainian (Ukraine)
zh-Hans	Chinese (Simplified)
zh-Hant	Chinese (Traditional)

Accessing the plugin in a game project

This plugin offers different ways for developers to access the underlying XeSS 2 functionality.

UE Console Commands

> It is more convenient to use for testing during development, not recommended for shipping.

XeSS-SR

To enable it:

```
r.XeSS.Enabled 1
```

To change the Quality Mode:

```
r.XeSS.Quality <quality_mode>
```

Where `<quality_mode>` represents the scale factor:

Value	Quality Mode	Scale factor
0	Ultra Performance	3
1	Performance	2.3
2	Balanced (default)	2
3	Quality	1.7
4	Ultra Quality	1.5
5	Ultra Quality Plus	1.3
6	Anti-Aliasing	1

Auto exposure, which is enabled by default.

```
r.XeSS.AutoExposure 1
```

> For more details on exposure calculation, please check `Intel® XeSS Developer Guide` in the `Documents` folder.

XeSS-FG

To enable it:

```
r.XeFG.Enabled 1
```

XeLL

To enable it:

```
r.XeLL.Enabled 1
```

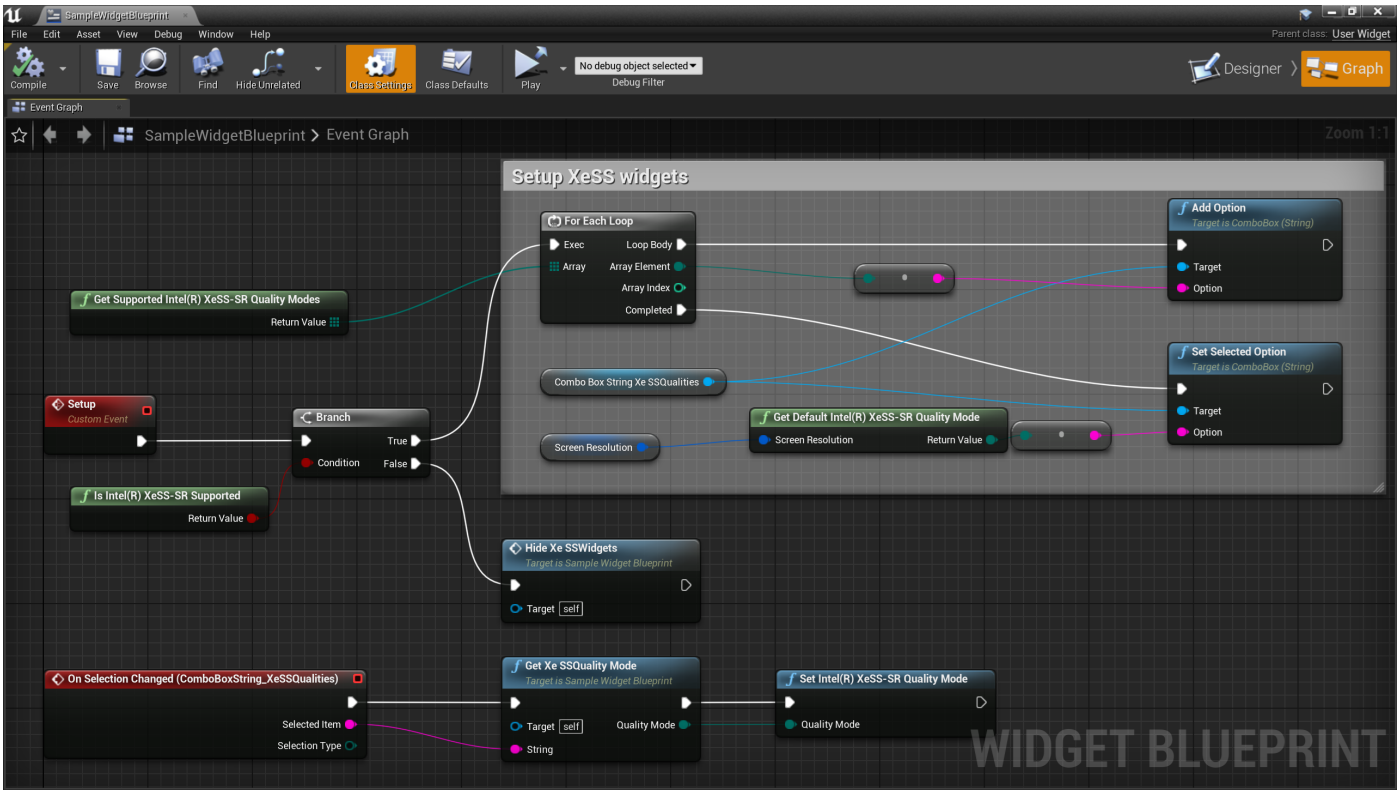
Blueprint API

Blueprint offers the highest level of compatibility and is the recommended way for shipping.

XeSS-SR Blueprint API

This plugin provides support for querying and settings XeSS-SR quality modes with Blueprint. It is recommended to use these functions when creating settings menus.

- *Is Intel(R) XeSS-SR Supported*
- *Get Supported Intel(R) XeSS-SR Quality Modes*
- *Get Current Intel(R) XeSS-SR Quality Mode*
- *Get Default Intel(R) XeSS-SR Quality Mode*
- *Set Intel(R) XeSS-SR Quality Mode*



Notes:

1. It is recommend to use the *Get Default Intel(R) XeSS Quality Mode* Blueprint function to set the out-of-the-box XeSS Quality Mode in the game's UI. Based on the passed Screen Resolution the recommended Quality Mode will be returned - for resolutions with pixel counts corresponding to 1920x1080 and lower it will be *Balanced*, for higher resolutions it will be *Performance*.

2. For XeSS-SR, Blueprint API is recommended to use in Blueprint or C++, for plugin loses control to upscaling screen percentage directly since UE 5.1 and `r.ScreenPercentage` is set in Blueprint API to keep its backward compatibility.

XeSS-FG Blueprint API

- *Is Intel(R) XeSS-FG Supported*
- *Get Supported Intel(R) XeSS-FG Modes*
- *Get Current Intel(R) XeSS-FG Mode*
- *Set Intel(R) XeSS-FG Mode*

XeLL Blueprint API

- *Is Intel(R) XeLL Supported*
- *Get Supported Intel(R) XeLL Modes*
- *Set Intel(R) XeLL Mode*
- *Get Current Intel(R) XeLL Mode*
- *Get Flash Indicator Enabled*
- *Get Game to Render Latency*
- *Get Game Latency*
- *Get Render Latency*
- *Get Simulation Latency*
- *Get Render Submit Latency*
- *Get Present Latency*
- *Get Input Latency*
- *Get Latency Mark Enabled*

C++ API

XeSS-FG C++ API

Similar to `GAverageFPS`, `GXeFGAverageFPS` is offered to display FPS counter in-game, sample code fragment as follows:

```
#include "XeFGRHI.h"

// Draw the FPS counter.
Canvas->DrawShadowedString(
    X,
    Y,
    *FString::Printf(TEXT("%5.2f FPS"), GXeFGAverageFPS),
    Font,
    FPSColor
);
```

Debugging

Querying SDK version

Feature	Console Command
XeSS-SR	`r.XeSS.Version`
XeSS-FG	`r.XeFG.Version`
XeLL	`r.XeLL.Version`

Querying feature support on current platform

Feature	Console Command
XeSS-SR	`r.XeSS.Supported`
XeSS-FG	`r.XeFG.Supported`
XeLL	`r.XeLL.Supported`

The support status depends on OS, RHI and Unreal Engine version. Please check FAQ for more detailed information.

Verifying if a feature is enabled in a running game

The easiest way to confirm that XeSS-SR or XeSS-FG are enabled is via:

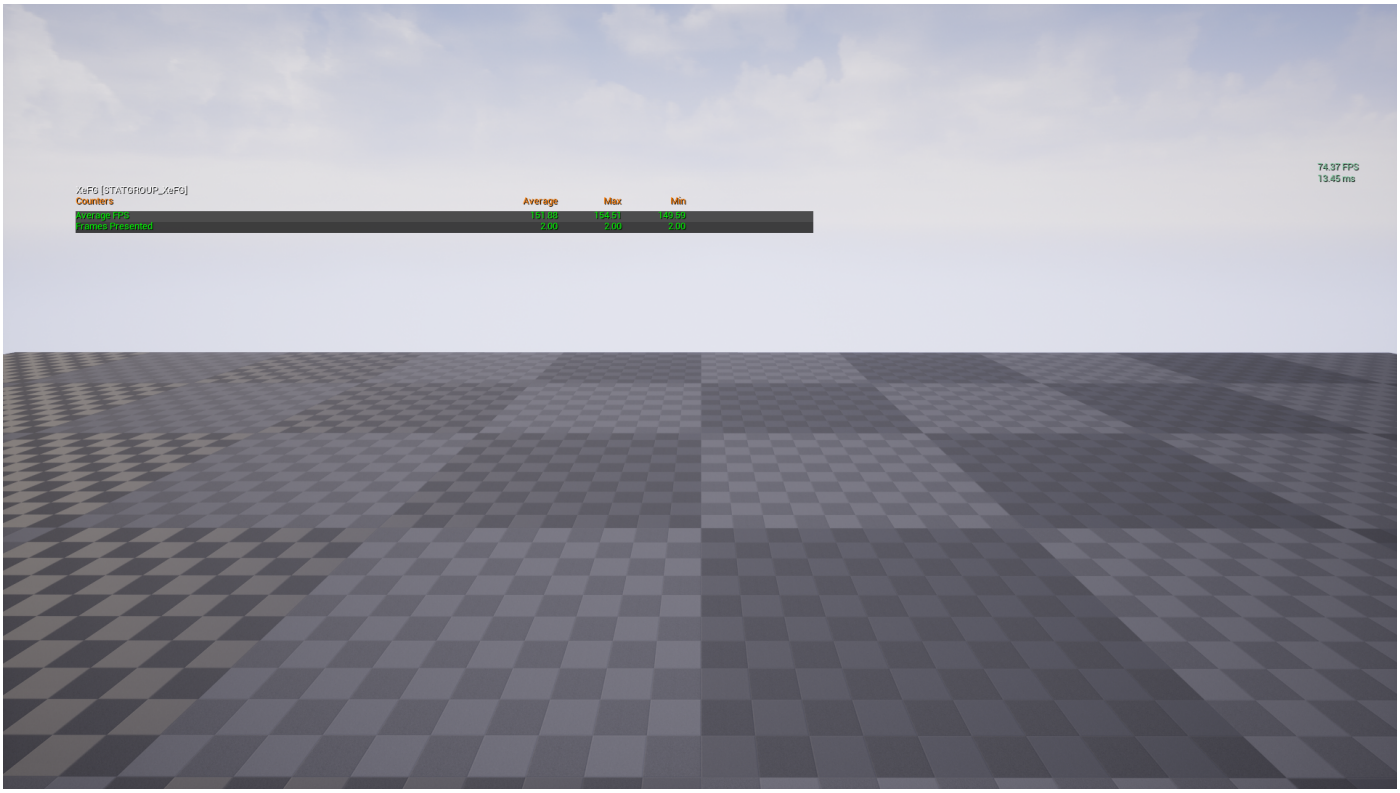
```
stat gpu
```

This will bring up real-time per-frame stats. `XeSS` or `XeFG` should be visible as one of the rendering passes.

For XeSS-FG, a dedicated stat console command is offered:

```
stat XeFG
```

You can use it to check average FPS and frame count presented with XeSS-FG.



Creating frame dumps

Frame dump console variables have been deprecated with XeSS 2, please use Intel® XeSS Inspector instead.

Using the High Resolution Screenshot tool

UE provides a console command that allows to take high resolution screen captures:

```
HighResShot <screen_resolution>
```

When using this tool while XeSS-SR is engaged please make sure to set *screen_resolution* to the currently set output resolution (can be set with *r.SetRes <screen_resolution>*). Otherwise, the capture tool will change the target resolution, which will re-initialize the XeSS context and drop all temporally accumulated data. In result the captured image will not reflect the actual quality seen on the screen.

FAQ

Platforms supported

Currently, only Windows x64 is supported.

Rendering Hardware Interfaces (RHIs) supported

Feature	RHIs Supported
XeSS-SR	DirectX 12
XeSS-FG	DirectX 12
XeLL	DirectX 12

Unreal Engine versions supported

Feature	UE Versions Supported
XeSS-SR	4.26 and above
XeSS-FG	5.2 and above, 4.27-5.1*
XeLL	4.27 and above

> *) For XeSS-FG with UE 4.27 and 5.0, 5.1, UE source code patch is required.
>
> 1. Contact your Intel representative to obtain the source code patch.
> 2. Apply the patch to the engine source code.
> 3. Modify `XeSSCommonMacros.h` with the following code if it is not defined in your source patch:

```
#define XESS_ENGINE_WITH_XEFG_PATCH 1
```

Relationship between XeSS-FG and XeLL

XeSS-FG requires XeLL to function. Enabling XeSS-FG will also enable XeLL, but disabling XeSS-FG will not disable XeLL, which is intentional.

Compatibility with other vendors' frame generation plugins

For XeSS-SR and XeLL, there should be no conflict.

For XeSS-FG, only one plugin could take effect due to swap chain override being required. You can disable it via following options and restart the game to make another plugin work.

1. To set the console variable `r.XeFG.OverrideSwapChain` to `0` via a ini file. e.g., add the following lines to the `Engine.ini` file:

```
[SystemSettings]
r.XeFG.OverrideSwapChain=0
```

2. Start the game with the command line option `-XeFGOverrideSwapChainDisabled`.

> **Note**: Using this command line option will also set the console variable `r.XeFG.OverrideSwapChain` to `0`.

Performance is not as good as expected on Intel discrete GPUs

It could be due to incorrect BIOS settings. Please ensure the following configurations are applied:

- Compatibility Support Module (CSM) or Legacy Mode: **Disabled**
- UEFI Boot Mode: **Enabled**
- The following settings must be **Enabled** (or set to **Auto** if the **Enabled** option is unavailable):
  - Above 4G Decoding
  - Resizable BAR Support

You can use the Intel® Driver and Support Assistant to verify if Resizable BAR Support is enabled.

Important Note:

Resizable BAR is not the only necessary BIOS setting; all of the above configurations are required. If any are improperly set, it may negatively impact GPU performance.

For more details, visit Intel's Support Article.

FPS drops after enabling XeSS-FG, what's wrong?

It may happen if you check FPS with `stat FPS` console command, because XeSS SDK calls additional `Present` API, which is not counted by `stat FPS`. To check the real FPS number, you could use console command `stat XeFG` or use Windows Game Bar via pressing Windows logo key + G.

Known issues

- Stereo rendering for VR or split screen usage is currently not supported.
- Pre-built UE 5.1 package doesn't work with 5.1.0, please upgrade to 5.1.1.
- If following link error occurs with pre-built packages, please upgrade Visual Studio to the latest version.

```
error LNK2019: unresolved external symbol __std_find_trivial_4 referenced in function "int *
__cdecl __std_find_trivial<int,char>(int *,int *,char)" (??
$__std_find_trivial@HD@@YAPEAHPEAH0D@Z)
```

- Pre-exposure is still not used in the exposure calculation process.
- XeSS-SR in Vulkan may not function properly on non-Intel GPUs.
- XeSS-FG doesn't support Editor yet, please use Standalone mode.
- XeSS-FG doesn't support fullscreen exclusive mode and will be disabled in that mode.